x

Foundations of Programming

Lab Assignment No. : 02

PROBLEM STATEMENT:

Write a menu driven program in C to implement the basic arithmetic operations such as Addition, Subtraction, Multiplication, and Division.

OBJECTIVES:

• To understand the concept of menu driven programming in C.

• To apply conditional statements and switch–case control structures.

• To perform basic arithmetic operations using user-defined input.

• To enhance logical thinking and problem-solving skills.

THEORY:

1. if–else ladder statements syntax and sample program

Conditional statements are used in C programming to **make decisions**.
They allow the program to **execute different blocks of code based on a condition** (true or false).
A] **If statement**: The **if statement** is a decision-making statement in C.
It is used to execute a block of code **only when a given condition is true**.
If the condition is false, the statements inside the `if` block are skipped

Syntax:

```
if (condition)
{
   statement(s);
```

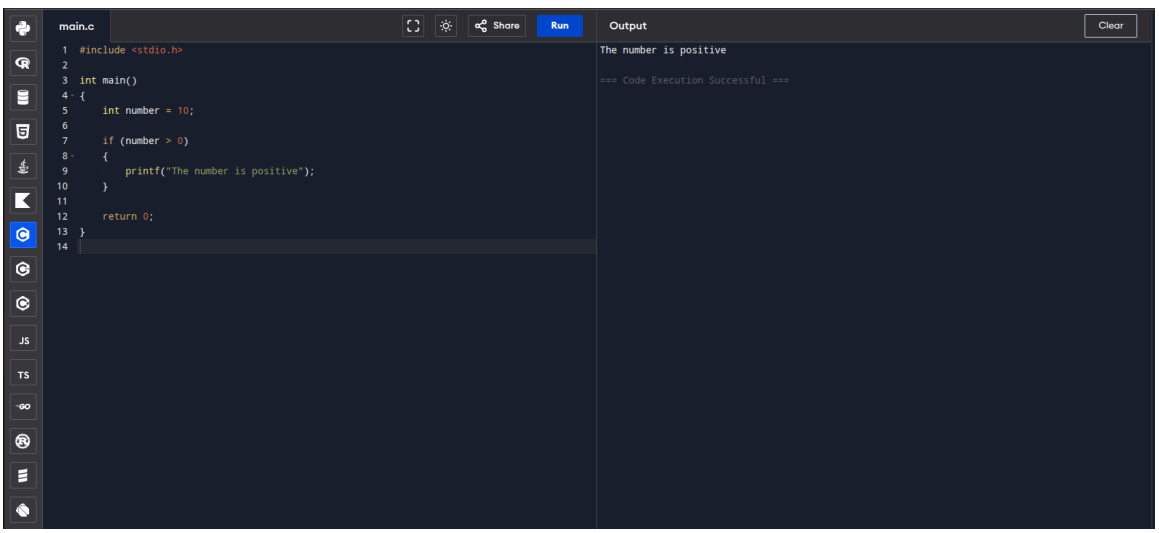
```
}
```

Program:

```
#include <stdio.h>

int main()
{
    int number = 10;

    if (number > 0)
    {
        printf("The number is positive");
    }

    return 0;
}
```

Output:

The number is positive



B] **<u>If-else statement:</u>** The **if–else statement** is used when there are **two possible outcomes**.

- If the condition is **true**, the `if` block is executed.
- If the condition is **false**, the `else` block is executed.

This ensures that **one block of code always executes**.

Syntax:

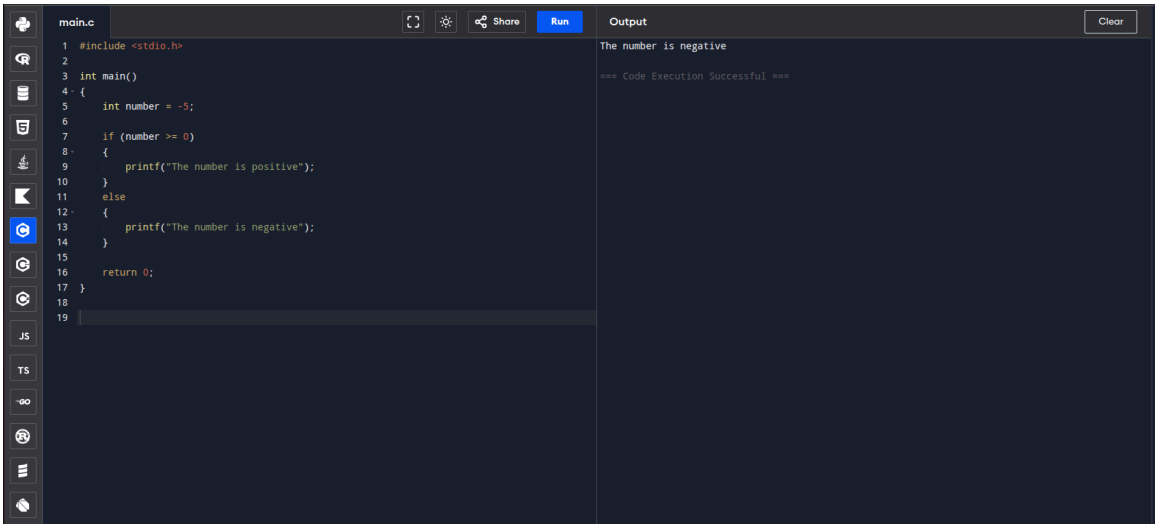
```
if (condition)
{
   statement(s);
}
else
{
   statement(s);
}
```

Program:

```
#include <stdio.h>

int main()
{
   int number = -5;

   if (number >= 0)
   {
      printf("The number is positive");
   }
   else
   {
      printf("The number is negative");
   }

   return 0;
}
```

Output:

The number is negative



C] **<u>If-else ladder</u>**: The **if–else ladder** is used when **multiple conditions** need to be checked.
Conditions are evaluated **from top to bottom**.

- When a true condition is found, its block is executed.
- Remaining conditions are skipped.
- If none of the conditions are true, the `else` block executes.

<u>Syntax:</u>

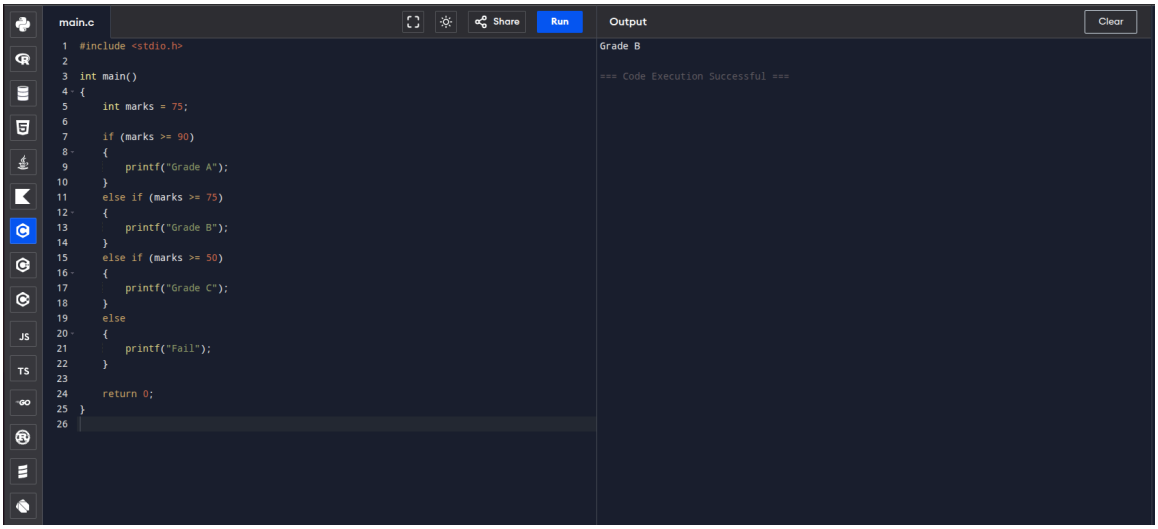
```
if (condition1)
{
   statement(s);
}
else if (condition2)
{
   statement(s);
}
else if (condition3)
{
   statement(s);
}
else
{
   statement(s);
```

}

Program:

```
#include <stdio.h>

int main()
{
    int marks = 75;

    if (marks >= 90)
    {
        printf("Grade A");
    }
    else if (marks >= 75)
    {
        printf("Grade B");
    }
    else if (marks >= 50)
    {
        printf("Grade C");
    }
    else
    {
        printf("Fail");
    }

    return 0;
}
```

Output : Grade B



2. switch–case or if–else ladder statement syntax and sample progrM

A] Switch Case

Syntax:

```
switch(expression)
{
   case constant1:
      // statements;
      break;

   case constant2:
      // statements;
      break;

   case constant3:
      // statements;
      break;

   default:
      // statements;
}
```

Program:

```
#include <stdio.h>


int main()
{
   int day;


   printf("Enter day number (1-7): ");
   scanf("%d", &day);


   switch(day)
   {
      case 1:
         printf("Monday");
         break;


      case 2:
         printf("Tuesday");
         break;


      case 3:
         printf("Wednesday");
         break;


      case 4:
         printf("Thursday");
```

```
            break;

        case 5:
            printf("Friday");
            break;

        case 6:
            printf("Saturday");
            break;

        case 7:
            printf("Sunday");
            break;

        default:
            printf("Invalid day number");
    }

    return 0;
}
```

Output:

```
Enter day number (1-7): 4
Thursday

=== Code Execution Successful ===
```

B] **If-else ladder**: The **if–else ladder** is used when **multiple conditions** need to be checked.
Conditions are evaluated **from top to bottom**.

- When a true condition is found, its block is executed.
- Remaining conditions are skipped.
- If none of the conditions are true, the `else` block executes.

Syntax:

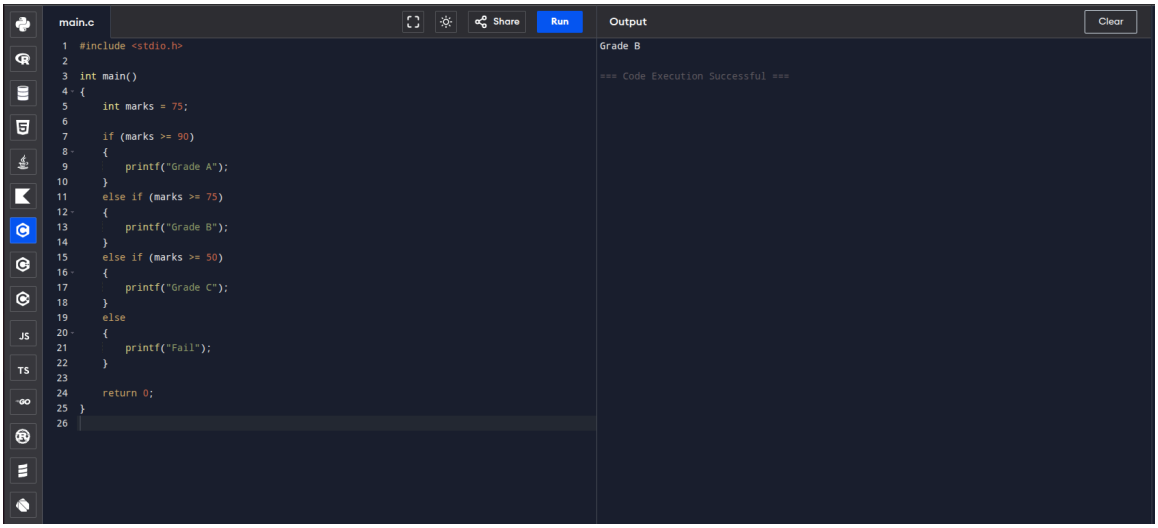
```
if (condition1)
{
   statement(s);
```

```
}
else if (condition2)
{
   statement(s);
}
else if (condition3)
{
   statement(s);
}
else
{
   statement(s);
}
```

Program:

```
#include <stdio.h>

int main()
{
   int marks = 75;

   if (marks >= 90)
   {
      printf("Grade A");
   }
   else if (marks >= 75)
   {
      printf("Grade B");
   }
   else if (marks >= 50)
   {
      printf("Grade C");
   }
   else
   {
      printf("Fail");
   }

   return 0;
}
```

Output : Grade B



Basic arithmetic operations include Addition, Subtraction, Multiplication, and Division. Division by zero should be avoided.

ALGORITHM:

```c
#include <stdio.h>

int main()
{
    int choice;
    float a, b, result;

    printf("Arithmetic Operations Menu \n");
    printf("1. Add\n");
    printf("2. Sub\n");
    printf("3. Multiply\n");
    printf("4. Divide\n");
    printf("Select one of the above (1-4): ");
    scanf("%d", &choice);

    printf("Enter first number: ");
    scanf("%f", &a);

    printf("Enter second number: ");
    scanf("%f", &b);

    switch (choice)
    {
        case 1:
            result = a + b;
            printf("\nAddition = %.2f", result);
            break;

        case 2:
            result = a - b;
            printf("\nSubtraction = %.2f", result);
            break;
```

```
        case 4:
            if (b != 0)
            {
                result = a / b;
                printf("\nDivision = %.2f", result);
            }
            else
            {
                printf("Error! Division by zero is not allowed.");
            }
            break;

        default:
            printf("Invalid choice! Please select between 1 and 4.");
    }

    return 0;
}
```

Type Program

```
#include <stdio.h>


int main()
{
   int choice;
   float a, b, result;


   printf("\n ---Arithmetic Operations Menu ---\n");
   printf("1. Add\n");
   printf("2. Sub\n");
   printf("3. Multiply\n");
   printf("4. Divide\n");
```

```
printf("Select one of the above (1-4): ");
scanf("%d", &choice);

printf("Enter first number: ");
scanf("%f", &a);

printf("Enter second number: ");
scanf("%f", &b);

switch (choice)
{
   case 1:
      result = a + b;
      printf("\nAddition = %.2f", result);
      break;

   case 2:
      result = a - b;
      printf("\nSubtraction = %.2f", result);
      break;

   case 3:
      result = a * b;
      printf("\nMultiplication = %.2f", result);
      break;
```

```
    case 4:
      if (b != 0)
      {
        result = a / b;
        printf("\nDivision = %.2f", result);
      }
      else
      {
        printf("Error! Division by zero is not allowed.");
      }
      break;

    default:
      printf("Invalid choice! Please select between 1 and 4.");
  }

  return 0;
}
```

OUTPUT:

Displays the result of the selected arithmetic operation

```
Arithmetic Operations Menu
1. Add
2. Sub
3. Multiply
4. Divide
Select one of the above (1-4): 1
Enter first number: 5
Enter second number: 6

Addition = 11.00

=== Code Execution Successful ===
```

PLATFORM:

Linux

INPUT:

Choice of operation and two numbers

SAMPLE INPUT:

5

6

SAMPLE OUTPUT:

Addition = 11

CONCLUSION:

Thus, the menu driven program in C successfully performs basic arithmetic operations using switch–case statements.

FAQs:

1. What is a menu driven program?

2. Why is switch–case preferred over if–else ladder in menu driven programs?

3. What is the purpose of the break statement in switch–case?

4. What is the purpose of the break statement in switch–case?

5. What are the limitations of switch–case compared to if–else?